kaiv 1.1

kaiv.io

Version 1.1 RC5 (2026-03-27)

Abstract

kaiv (pronounced: "cave") is a line-oriented data format whose syntax is defined by a *regular grammar*. Because every line can be classified by a deterministic finite automaton in a single left-to-right scan, kaiv parsers are structurally simple: no lookahead, no backtracking, and no recursion. (see Section 1.1 for the precise meaning of "parser" in kaiv terminology).

This structural simplicity is achieved by a deliberate trade-off: kaiv has no escape sequences, no nesting, no multiline values at the syntax level, and no variable interpolation. These are not missing features — they are excluded by design to keep the grammar regular. The resulting constraint is that kaiv parsers need not maintain a stack or a tree, and can emit entries as each line is scanned.

The kaiv architecture draws a sharp boundary between two concerns:

- **Parsing** (this specification): reading kaiv text and emitting an entry stream — a sequence of key-value pairs, array entries, metadata entries, and comment entries, each tagged with a line number. Parsers perform syntactic validation (UTF-8 well-formedness, key patterns, directive recognition) but do not interpret values.
- **Application logic** (outside this specification): consuming the entry stream and applying semantic interpretation. This includes, but is not limited to: variable interpolation, type casting, schema validation, default values, environment-specific overrides, and duplicate key resolution.

This separation means that features commonly associated with configuration file formats — such as `${VAR}` expansion, conditional logic, or include directives — are application-level concerns, not parser-level syntax. Nothing in this specification prevents applications from implementing any of these; the specification simply does not prescribe them, leaving implementations free to adopt whatever strategy suits their domain. (A subset of the POSIX variable syntax is mentioned in the Appendix as a recommendation.) File composition (includes, overlays, conditional sections) is addressed by the kaiv preprocessor, a separate specification.

kaiv is defined at the specification level, not by a reference implementation. The format is conceived abstractly so that many different implementations can coexist: a streaming parser for embedded systems, a channel-based parser in Go or Erlang, a DOM-building library for desktop applications, an iterator-based parser in Rust or Python. Conformance is determined by adherence to this specification, not by behavioral equivalence with any particular implementation.

This document specifies kaiv version 1.1, which extends the 1.0 foundation (`.env` standardization) with: a format declaration and version system, quoted keys, schemas (relaxed and strict), type annotations and custom annotations, a base64url encoding directive, a newline alias directive for representing multiline values, arrays with explicit separators, and C-style line comments (`//`) for international keyboard accessibility. All kaiv 1.0 texts are valid kaiv 1.1 texts.

Table 1: Document Status

Field	Value
Version	1.1 RC5
Publication Date	2026-03-27
Status	Under Review
Planned Stable	2026-06-15
Editor	Bojan Đuričković
Email	v1.1-rc5@kaiv.io

Contents

I .env Standard (v1.0) 6

1 Introduction 6
1.1 Scope . . . 6
1.2 Specification Structure . . . 7

2 Terminology 7
2.1 Containers . . . 7
2.2 Atomic Building Blocks . . . 7
2.3 Text Elements . . . 7
2.4 Entries . . . 8
2.5 Whitespace . . . 8
2.6 Line and EOL . . . 8
2.7 Parser and Application . . . 8

3 Syntax 8
3.1 Data Line . . . 9
3.2 Comment Line . . . 9
3.3 Blank Line . . . 10
3.4 EOL . . . 10
3.5 Shebang Line (File-Level) . . . 10

4 Parsing 10
4.1 Line Numbers . . . 10
4.2 Entry Structure . . . 11
4.3 Entry Stream . . . 11
4.4 General Parsing Requirements . . . 11
4.5 Whitespace Handling . . . 11
4.6 Value Preservation . . . 12
4.7 UTF-8 Processing . . . 12
4.8 Error Handling . . . 12
4.9 Implementation Limits . . . 12

5 Semantics 13
5.1 Values are Verbatim Literals . . . 13
5.2 Duplicate Key Semantics . . . 13

6 File Representation **13**
6.1 File Extension . . . 13
6.2 Media Type (MIME Type) . . . 13
6.2.1 Choosing Between `application/kaiv` and `text/kaiv` . . . 14
6.2.2 Transport Considerations for `text/kaiv` . . . 14
6.2.3 Parameters . . . 14
6.2.4 Examples . . . 15
6.2.5 Future Registration . . . 15
6.3 File Encoding . . . 15
6.4 Shebang Lines . . . 15

7 Examples **16**
7.1 Valid Examples . . . 16
7.2 Invalid Examples . . . 17
7.3 Edge Cases . . . 17

8 Versioning **18**

II Basic kaiv Syntax **18**

9 Format Declaration **18**
9.1 General Format Declaration Syntax . . . 18
9.2 Version Declaration Syntax . . . 19
9.3 Version Declaration Errors . . . 19

10 Quoted Keys **19**
10.1 Quoted Key Syntax . . . 19
10.2 Examples . . . 20

11 kaiv Comment Syntax **20**

III Metadata **20**

12 Metadata Entries **20**

13 Type Annotations **21**
13.1 Syntax . . . 21
13.2 Semantics . . . 21
13.3 Standard Type Annotations . . . 21
13.4 Custom Type Annotations . . . 22
13.5 Parsing Requirements . . . 22
13.6 Application Behavior . . . 22

14 Encoding Directive **22**
14.1 Syntax . . . 22
14.2 Base64url Variant . . . 23
14.3 Examples . . . 23
14.4 Parsing Requirements . . . 23
14.5 Application Behavior . . . 23

15 Newline Alias Directive **23**
15.1 General Syntax . . . 24
15.2 Multi-character newline aliases . . . 24
15.3 Newline Alias Selection . . . 24
15.4 Parsing Requirements . . . 25
15.5 Application Behavior . . . 25

16 Metadata Forward Compatibility **25**

IV Arrays **25**

17 Arrays **25**
17.1 Array Syntax . . . 26
17.2 Whitespace Handling . . . 26
17.3 Array Entries . . . 27
17.4 Array Parsing Requirements . . . 27

18 Arrays and Directives **27**

V Schemas **27**

19 Schema Definition **28**
19.1 Schema Format Declaration . . . 28
19.2 Schema Field Definitions . . . 28
19.3 Schemas and Key Uniqueness . . . 29

20 Schema Declaration in Data Texts **29**

21 Relaxed and Strict Schemas **29**

22 Schema Composition and Inheritance **30**
22.1 Schema Composition . . . 30
22.2 Schema Inheritance . . . 30

23 Schemas and Metadata **30**
23.1 Type Annotations and Schemas . . . 30
23.2 Encoding Directives and Schemas . . . 31
23.3 Newline Alias Directives and Schemas . . . 31

24 Schemas and Arrays **31**

25 Schema Constraints **31**
25.1 Constraints Syntax . . . 32
25.2 Constraints and Arrays . . . 32
25.3 Ordering and Comparison . . . 32
25.4 Parsing Requirements . . . 33
25.5 Application Behavior . . . 33

26 References **33**

Appendices **34**

Appendix A Formal Grammar **34**
A.1 Grammar . . . 34
A.2 Notes . . . 35
A.3 Schema Definition . . . 35

Appendix B Parser Errors **36**
B.1 `BOM_ERROR` . . . 36
B.2 `INVALID_UTF8_ERROR` . . . 36
B.3 `INVALID_VERSION_ERROR` . . . 36
B.4 `UNSUPPORTED_VERSION_ERROR` . . . 36
B.5 `INVALID_CHARACTER_ERROR` . . . 36
B.6 `EMPTY_KEY_ERROR` . . . 37
B.7 `MISSING_OPERATOR_ERROR` . . . 37
B.8 `INVALID_KEY_ERROR` . . . 37
B.9 `INVALID_DIRECTIVE_ERROR` . . . 37
B.10 `INVALID_SEPARATOR_ERROR` . . . 37
B.11 `INVALID_NL_ALIAS_ERROR` . . . 37
B.12 `INVALID_CONSTRAINT_ERROR` . . . 37
B.13 `MISSING_FINAL_EOL_ERROR` . . . 38

Appendix C Application Errors **38**
C.1 `MetadataWithoutTargetError` . . . 38
C.2 `SchemaDuplicateKeyError` . . . 38
C.3 `SchemaResolutionError` . . . 38
C.4 `DuplicateKeySchemaError` . . . 38
C.5 `RequiredKeySchemaError` . . . 38
C.6 `UndefinedFieldStrictSchemaError` . . . 38

Appendix D Variable Interpolation (Non-Normative) **39**
D.1 Parser Behavior . . . 39
D.2 Recommended Syntax . . . 39
D.3 Variable Resolution . . . 40
D.4 Interaction with Encoding Directives . . . 40
D.5 Interaction with Newline Alias Directives . . . 40
D.6 Interaction with Arrays . . . 40
D.7 Schemas and Variable Interpolation . . . 40

Part I
.env Standard (v1.0)

1 Introduction

The driving idea of kaiv (pronounced: "cave") is to use a *regular grammar*— a grammar class strictly below the context-free grammars of JSON and TOML in the Chomsky hierarchy.

It builds upon .env file conventions while imposing constraints necessary for regular language properties. Thus, kaiv 1.0 is a standardized minimal common denominator to .env files used in practice, intended to provide a foundation for extensions in 1.x versions.

The goal of kaiv is to provide:

1. *a regular grammar*, parsable with deterministic finite automata where each character is examined exactly once, with:
   - *a minimal syntax* with no escape mechanisms, no multiline values, and no interpolation,
   - *a compact syntax* with no structural whitespace, and no significant quotes or brackets,
2. *a streaming-first design*, with entry stream enabling CSP/channel-based processing,
3. *application flexibility*, supporting different duplicate key handling strategies, and optional comment preservation.

This design trades expressiveness (escaping, nesting, interpolation, multiline values) in exchange for single-pass parsing, deterministic processing, and straightforward implementation.

To avoid confusion with incompatible `.env` dialects, kaiv files should use the `.kaiv` extension.

1.1 Scope

This specification defines:

- kaiv text syntax,
- parsing requirements, including error conditions,
- the *entry stream* — the sequence of parsed entries that parsers emit (conceptual definition only).

This specification does *not* define:

- parser APIs — implementations may use channels, iterators, callbacks, or other patterns;
- entry processing — applications may consume entries in any suitable way;
- data structures — internal representation of parsed or processed data.

In kaiv terminology, the *parser* processes kaiv text to emit *entries*. The parser's responsibility ends with entry emission; application logic that consumes entries is outside this specification's scope. Thus, a parser in kaiv terminology has a narrower meaning than typical: it is a token producer that yields entries for subsequent processing, without constructing concrete or abstract syntax trees.

A technically precise term for what this specification calls a "parser" would be a *lexer* or a *scanner* (see e.g., [1]). We use "parser" because it is the established term in data format specifications and avoids implying a subsequent parsing stage. Comparisons with parsers of context-free formats (JSON, TOML, YAML) should be understood in this light: those parsers build recursive structures, whereas a kaiv parser performs lexical analysis only, emitting a flat entry stream.

The *entry stream* abstraction separates parsing (syntax recognition) from processing (semantic interpretation). This enables flexible integration into different application architectures while maintaining a clear boundary between the parser's responsibilities and the application's domain logic.

1.2 Specification Structure

This specification is structured for stable evolution across 1.x versions:

- Sections — *Normative Requirements*: define the stable syntax, parsing model, and requirements of kaiv. The intention is to preserve Sections unchanged in kaiv 1.x versions, with new sections appended as needed.
- Appendices — *Technicalia*: contain formal definitions, algorithms, and implementation details that may be refined in minor versions while maintaining compatibility with the Normative Requirements.

The Technicalia provide the authoritative interpretation of the Normative Requirements. Implementations that conform to the Technicalia necessarily satisfy the Normative Requirements, and in case of ambiguity, the Technicalia has precedence.

2 Terminology

The all-caps key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119 [2].

Unicode code points in the U+XXXX form are defined in the Unicode Standard [10].

2.1 Containers

- **kaiv text**: a Unicode string encoded in UTF-8 that conforms to the kaiv grammar
- **kaiv file**: a kaiv text stored in a file system, optionally preceded by a shebang line

2.2 Atomic Building Blocks

- **key**: a string identifier, a variable name [1]
- **value**: a single-line string

2.3 Text Elements

A kaiv text is composed of different types of lines:

- **data line**: a line defining a key-value data entry, consisting of:
  - a *key*,
  - the **assignment operator**,

[1] In the standard form (kaiv 1.0), keys must be valid POSIX variable names with no reservations. UTF-8 keys are introduced in kaiv 1.1 as a Unicode variant (kaiv 1.1u). See Appendix A for details.

  - a *value*;

- **comment line**: a line defining a comment entry, starting with `#` and extending to the end of the line;
- **blank line**: an empty line.

Data lines and *comment lines* are collectively termed **content lines**.

2.4 Entries

The parser emits an output of parsed **entries**, which come in several varieties:

- **KV entry**: a key-value pair of strings that contributes to the document's data (always emitted),
- **comment entry**: string annotation that does not contribute to the document's data but may be preserved by the parser for round-trip serialization (optional),
- **blank entry**: result of parsing a blank line (optional),
- **shebang entry**: a representation of a shebang line (optional).

2.5 Whitespace

In this specification, **whitespace** refers exclusively to:

- `U+0020 SPACE`
- `U+0009 CHARACTER TABULATION`

In kaiv syntax, whitespace characters are generally treated like any other characters, except that they are tolerated as indentation and as optional spacing between keys and assignment operators.

2.6 Line and EOL

EOL (End of Line) is a line terminator, either LF (POSIX-style line feed) or CRLF (Windows-style line ending). Intermixing of LF and CRLF is tolerated within the same kaiv text.

In this specification, a **line** refers to the sequence of characters from the start of the line up to and *including* the line terminator (EOL).

2.7 Parser and Application

A **parser** is a process that reads a kaiv text and emits an **entry stream** according to this specification.

An **application** is a process that consumes the entry stream produced by a parser. Applications are responsible for semantic interpretation of entries, including but not limited to duplicate key handling (Section 5.2) and value interpretation. Starting with kaiv 1.1, application responsibilities extend to schema validation, metadata processing, and associated error conditions.

3 Syntax

This section represents a descriptive definition of the kaiv syntax. The formal grammar presented in Appendix A is the normative definition, and has precedence in case of any ambiguity.

3.1 Data Line

A key-value data line has the basic form (BNF):

```
key OP value EOL
```

where:

- `key` is a string identifier,
- `OP` is an assignment operator; for simple string values this is `%x3D; EQUALS SIGN` (`=`),
- `value` is the remainder of the line, extending from the assignment operator to the end of the line, excluding the EOL,
- `EOL` is the line terminator (LF or CRLF).

For example:

```
host=kaiv.io
```

Keys and assignment operators may be indented for presentation clarity. For example:

```
    database =db1
    port     =5432
    user     =admin
```

Note that any whitespace characters *after* the assignment operator `=` would *not* be considered optional indentation, but part of the value.

The general ABNF form of a data line is therefore

```
*WSP key *WSP OP value EOL
```

where `*WSP` represents zero or more whitespace characters.

3.2 Comment Line

A comment line has the basic form (BNF):

```
"#" comment-text EOL
```

where `#` is the `NUMBER SIGN` character (`%x23`).

For example:

```
# This is a comment
```

As with data lines, comments may be indented. The following lines are all syntactically equivalent:

```
# comment
 # comment
    # comment
```

This does *not* hold for whitespace characters *after* the `#` character, which are considered part of the comment text. Thus, the following lines are all different:

```
#comment
# comment
#    comment
```

General ABNF form of a comment line:

```
*WSP "#" comment-text EOL
```

3.3 Blank Line

A blank line is an empty line or a line containing only whitespace characters:

```
*WSP EOL
```

Blank lines are used for visual separation without semantic meaning.

3.4 EOL

EOLs are line terminators.

```
EOL = LF / CRLF
```

where:

```
— LF = %x0A;LINE FEED
— CR = %x0D;CARRIAGE RETURN
— CRLF = CR LF
```

Every line in a kaiv text MUST be terminated with an EOL (either LF or CRLF), including the final line.[2] An EOL is considered part of the line (see Section 2.6).

3.5 Shebang Line (File-Level)

Shebang lines [8] are file-level directives, not part of kaiv text syntax. Shebangs MUST NOT be treated as comments. See Section 6.4 for details.

4 Parsing

A parser processes a kaiv text line by line, emitting a stream of entries.

4.1 Line Numbers

Parsers MUST keep track of line numbers.

- Line numbers MUST be 1-based.
- Parsers MUST include line numbers with entries.
- Parsers SHOULD reference line numbers in error reporting.

[2] The strict requirement on the final line including an EOL enables deterministic streaming parsing, where EOL serves as the definitive marker that a complete line has been received. Without a final EOL, streaming implementations cannot reliably distinguish incomplete transmissions from valid end-of-input.

4.2 Entry Structure

Each line is parsed into an entry that contains:

- *type*: implementation-defined means to distinguish between entry types (KV, comment, blank, or shebang),
- *line*: integer line number (Section 4.1),
- *payload*: type-specific data:
  - *KV entries*: two strings *(key, value)*,
  - *comment entries*: one string *(comment text)*,
  - *blank entries*: no payload,
  - *shebang entries*: one string *(line content)*.

Entry payloads contain the line's semantic content; the EOL terminator is consumed during parsing and is not part of any entry payload.

The method of representing entries (tuples, structs, objects, etc.) is implementation-defined.

Keys in KV entries MUST NOT contain any whitespace characters. Any leading or trailing whitespace in a key MUST be removed by the parser, while any whitespace inside the key represents an `INVALID_KEY_ERROR` (Appendix B.8).

4.3 Entry Stream

Parsers emit an **entry stream** — a unidirectional flow of entries derived from lines in the kaiv text.

- For each *data line*, a parser MUST emit a corresponding *KV entry*.
- For each *comment line*, a parser MAY emit a corresponding *comment entry*.
- For each *blank line*, a parser MAY emit a *blank entry*. If entries are emitted out of order (e.g., parallel parsing), blank entries MUST be emitted.

Shebang handling: When parsing a kaiv file that begins with a shebang line (see Section 6.4), the parser MAY emit a *shebang entry* for the shebang line. If entries are emitted out of order, the shebang entry MUST be emitted.

Shebang entries MUST preserve all characters of the shebang line verbatim except the EOL terminator, including the `#!` prefix. Shebang entries MUST NOT be emitted for any line beyond the first.

4.4 General Parsing Requirements

1. Parsers MUST parse texts matching the grammar defined in Appendix A.
2. Parsers MUST detect all error conditions in Appendix B.
3. Parsers MUST NOT handle duplicate key detection or processing. Duplicate key handling is an application-level decision.
4. The emitted entries MUST conform to the entry structure (Section 4.2) and entry stream requirements (Section 4.3).
5. Parsers MAY emit entries in any order. Implementations MUST document their emission order.

4.5 Whitespace Handling

1. Parsers MUST remove leading whitespace (any whitespace between the start of the line and the first non-whitespace character) in all lines.

2. Parsers MUST remove whitespace between keys and assignment operators.
3. Parsers MUST NOT remove any whitespace characters after the assignment operator.

4.6 Value Preservation

Parsers MUST treat all kaiv text values as strings, and MUST preserve values *verbatim*. Parsers MUST NOT interpret any characters (including but not limited to quotes, backslashes, dollar signs, percent signs, etc.) as having special meaning or initiating escape sequences. Specifically:

- quotes MUST be preserved as part of the value,
- backslashes MUST be preserved as literal backslash characters, etc.

Values MUST NOT contain EOL characters. All other characters, including any leading or trailing whitespace characters (see Section 4.5), MUST be preserved verbatim.

Empty value parsing: When an assignment operator is immediately followed by EOL, parsers MUST emit the value as an empty string.

4.7 UTF-8 Processing

Parsers MUST ensure valid UTF-8 encoding. Validation may occur:

- before parsing (whole text),
- during parsing (line by line), or
- while streaming.

BOM handling: The UTF-8 Byte Order Mark (BOM) is not supported by kaiv. If a kaiv text begins with the bytes `EF BB BF`, parsers MUST raise a `BOM_ERROR` (Appendix B.1).

4.8 Error Handling

Parsers implement one of two models:

1. *eager parsing*: validate entire text before emitting entries,
2. *streaming parsing*: emit entries as lines are parsed.

Parsers MUST document which model they implement and their behavior when error occurs:

- eager parsers: whether any entries are emitted in case of error,
- streaming parsers: whether entries before the first error are emitted, and whether parsing continues after errors.

Lines causing errors SHOULD NOT emit entries.

A comprehensive list of parsing errors is given in Appendix B.

4.9 Implementation Limits

This specification does not mandate maximum limits for:

- line length,
- number of entries in a document,
- key length,
- value length, or

— total document size.

Implementations MAY impose reasonable limits based on available resources, but SHOULD document these limits and provide clear error messages when limits are exceeded.

5 Semantics

By design, kaiv relegates most semantic choices to applications consuming the entry stream. This includes decisions such as value whitespace trimming, duplicate key handling, and comment preservation. This section is therefore limited to reiterating some basic principles.

5.1 Values are Verbatim Literals

All values in kaiv preserve all characters exactly as written (see Section 4.6). No characters have special meaning: quotes, backslashes, dollar signs, percent signs, etc., are all literal characters.

This design gives applications flexibility to interpret values according to their needs. For example, a configuration loader might trim trailing whitespace, while a round-trip serializer would preserve it exactly.

5.2 Duplicate Key Semantics

Consistent with .env file precedent, kaiv texts may contain duplicate keys. This follows from the regular grammar: parsers emit entries without maintaining key state.

The entry stream thus contains all instances of repeated keys. Applications consuming the entry stream determine how to handle duplicate keys according to their needs. Common strategies include:

— *first wins*: use the first occurrence, ignore subsequent values;
— *last wins*: use the last occurrence, overriding previous values;
— *concatenate*: join all occurrences into a single string;
— *preserve all*: keep all values as a list/array or in a multiset;
— *reject*: treat duplicates as error.

6 File Representation

6.1 File Extension

Files consisting of a kaiv text SHOULD use the `.kaiv` file extension.

6.2 Media Type (MIME Type)

With kaiv, we recognize that the same content may serve dual purposes: machine-to-machine data exchange and human-readable documentation. Following the precedent of XML (RFC 7303 [11]), kaiv defines two media types to accommodate this duality.

Target media types: Should kaiv gain sufficient adoption, the following media type conventions may serve as a basis for future IANA registration.

— `application/kaiv`
— `text/kaiv`

Provisional media types: Until IANA registration is obtained following RFC 6838 [6], implementations SHOULD use the provisional forms:

- `application/vnd.kaiv`
- `text/vnd.kaiv`

The remainder of this section uses the target names for clarity of discussion.

6.2.1 Choosing Between `application/kaiv` and `text/kaiv`

The choice depends on the intended use of the content:

Context	Recommended Type	Rationale
API payloads, configuration delivery, machine-to-machine, streaming pipelines	`application/kaiv`	Safe default; preserves exact byte-for-byte content; no risk of EOL normalization or stripping
Display in browsers, email, documentation systems, debugging output	`text/kaiv`	Signals human-readability; may enable inline display or syntax highlighting

When in doubt, use `application/kaiv`. It is the safer choice for preserving data integrity across transport layers.

6.2.2 Transport Considerations for `text/kaiv`

The `text/` top-level type carries inherent risks that senders MUST consider:

- *Line ending normalization:* Some transport systems (email gateways, older HTTP implementations) may convert line endings to local conventions (e.g., CRLF → LF or vice versa).
- *Final EOL stripping:* Some systems may remove the final EOL character, which would cause a kaiv parser to raise a `MISSING_FINAL_EOL_ERROR` (Appendix B.13).
- *Charset re-encoding:* Although kaiv requires UTF-8, `text/` types default to US-ASCII (RFC 2046 [5], Section 4.1.2) unless an explicit `charset` parameter is provided. This default can trigger charset conversions in legacy systems.

Senders SHOULD use `application/kaiv` when exact byte-for-byte preservation is required, or when the transport path is not fully trusted to preserve `text/` semantics.

6.2.3 Parameters

Both media types support the following optional parameters (following RFC 2045 syntax [4]):

- `version`: OPTIONAL. If present, the value MUST be a string (quoted or unquoted) containing the kaiv version number (e.g., `1.0`). This parameter indicates which version of the kaiv syntax the content conforms to.
- `charset`: The handling of this parameter differs by top-level type:
  - **`application/kaiv`:** OPTIONAL. If present, the value MUST be `utf-8`. Since kaiv requires UTF-8 encoding (Section 6.3), this parameter is redundant but MAY be included for compatibility with systems that require explicit charset declaration.

- **`text/kaiv`:** REQUIRED. The value MUST be `utf-8`. Because `text/` subtypes default to US-ASCII per RFC 2046, omitting this parameter may cause receivers to misinterpret non-ASCII content. Senders MUST include `charset=utf-8` when using the `text/kaiv` media type.

Both media types MUST be interpreted identically by kaiv parsers upon receipt.

6.2.4 Examples

Example HTTP Content-Type headers using target media types:

```
Content-Type: application/kaiv
Content-Type: application/kaiv; version=1.0
Content-Type: application/kaiv; version=1.0; charset=utf-8
Content-Type: text/kaiv; version=1.0; charset=utf-8
Content-Type: text/kaiv; charset=utf-8
```

Example HTTP Content-Type headers using provisional media types:

```
Content-Type: application/vnd.kaiv; version=1.0
Content-Type: text/vnd.kaiv; version="1.0"; charset="utf-8"
```

Note: `text/kaiv` without `charset=utf-8` is non-conformant:

```
Content-Type: text/kaiv; version=1.0
             <- WRONG: missing required charset
```

6.2.5 Future Registration

Should kaiv gain sufficient adoption, `application/kaiv` and `text/kaiv` may be submitted for IANA registration following RFC 6838 [6]. Until such time, implementations SHOULD use the provisional `application/vnd.kaiv` and `text/vnd.kaiv` types.

6.3 File Encoding

Any kaiv files MUST be encoded as UTF-8 without a Byte Order Mark (BOM).

6.4 Shebang Lines

A **shebang line** [8] is an optional first line in a kaiv file that specifies how to execute the file as a script. Shebang lines typically have the form:

```
#!/path/to/interpreter [optional argument]
```

- Shebang lines MUST start with the bytes `#!`.
- Shebang lines are system file-level directives and are not considered part of the kaiv text.
- When a kaiv file has a shebang line, parsers MAY emit *shebang entries* (Section 4.3) for round-trip serialization.
- When emitting shebang entries, parsers MUST preserve shebang lines *verbatim*. Normalization, trimming, or validation of shebang content MUST NOT be performed.

7 Examples

7.1 Valid Examples

1. Simple examples:

```
APP_NAME=My Application
API_KEY=sk_live_1234567890abcdef
DEBUG=true
PATH=/usr/local/bin:/usr/bin
```

2. Empty value (value is an empty string):

```
EMPTY=
```

3. Leading and trailing space in values preserved (values are: «`foo·`» and «`·bar`»)[3]

```
trailing=foo·
leading=·bar
```

4. No inline comments (value is «`1 # default value`»)

```
NOT_AS_INTENDED=1 # default value
```

5. Quotes are characters like any other (value is «`"hello"`»)

```
quoted="hello"
```

6. Backslashes are literals, never used as escapes (value is «`\n`» —two characters: a backslash and the letter "n")

```
BACKSLASH_N=\n
```

7. No interpolation (values are «`/usr/bin`» and «`$PATH:/usr/var/bin`»)

```
PATH=/usr/bin
PATH=$PATH:/usr/var/bin
```

8. Empty comment (comment text is an empty string)

```
#
```

9. Comment consisting of a space (comment text is «`·`»)

```
#·
```

10. Comment with leading and trailing spaces (comment text is «`··comment···`»)

```
#··comment···
```

11. Duplicate keys are valid in the parser context (see Section 5.2); this results in two emitted entries:

```
KEY=1
KEY=2
```

[3] Dots represent space characters.

7.2 Invalid Examples

1. Invalid key: dashes not allowed (`INVALID_KEY_ERROR`)

```
KEY-NAME=value
```

2. Invalid key: cannot start with a digit (`INVALID_KEY_ERROR`)

```
123KEY=value
```

3. Invalid key: cannot contain period (`INVALID_KEY_ERROR`)

```
KEY.NAME=value
```

4. Missing assignment operator `=` (`MISSING_OPERATOR_ERROR`)

```
KEY value
```

5. Empty key (`EMPTY_KEY_ERROR`)

```
=value
```

7.3 Edge Cases

1. An empty text (0 bytes) is a valid kaiv text and yields no entries.
2. Comments, not data lines

```
#=foo
#key=value
```

3. Valid data line with value «`=foo`».

```
key==foo
```

4. Valid data line with an empty string for a value

```
key=
```

5. Valid data line with a leading space in the value: «`·abc`»

```
key = abc
```

6. Error - space is not an operator (`MISSING_OPERATOR_ERROR`)

```
key value
```

7. Error - colon is not an operator (`MISSING_OPERATOR_ERROR`)

```
key:value
```

8. Error - colon is not a valid key character (`INVALID_KEY_ERROR`)

```
key:=value
```

9. Error - space is not a valid key character (`INVALID_KEY_ERROR`)

```
foo bar=value
```

10. Error: whitespace as key is an `EMPTY_KEY_ERROR`, not an `INVALID_KEY_ERROR` (leading whitespace may be trimmed before parsing the line)

```
·=value
```

8 Versioning

The kaiv 1.x versions are not sequential updates in the traditional sense. All 1.x format definitions were designed and specified concurrently, then organized into a layered specification collection with progressive feature inclusion. This approach reflects the fundamental trade-off between feature richness and implementation size.

The kaiv 1.x versions provide increasing functionality:

- 1.0 — basic key-value strings with ASCII-only keys (minimal .env file standard),
- 1.1 — metadata: schemas, type hints, encoding directives, and UTF-8 support for keys,
- 1.2 — arrays (support for sequences as values),
- 1.3 — namespaced keys,
- 1.4 — structs (support for nested key/value data as values).

Each version includes all features of previous versions. The complete 1.4 specification is not an "update" to 1.0 but a superset specification that adds new syntactic constructs while maintaining backward compatibility.

Because kaiv's syntax is a regular language, parsers require no recursion or stack, which may yield smaller binary footprints and lower per-line overhead compared to context-free parsers (JSON, TOML, YAML). Actual performance depends on the implementation language, the error reporting strategy, and the feature set (version) implemented.

The layered version approach directly addresses resource-constrained systems by enabling implementers to choose the minimal feature set their application requires:

- Applications needing only basic key-value strings (e.g., environment variables with ASCII keys) can implement kaiv 1.0, yielding a binary typically a fraction of the size of a full 1.4 implementation.
- Applications requiring Unicode keys or arrays but not namespaces or structs can implement kaiv 1.1, still significantly smaller than versions 1.2+.

Each successive version adds parsing complexity and increases the minimum implementation size. The version numbers thus serve as size/complexity indicators rather than "latest and greatest" markers.

Part II
Basic kaiv Syntax

9 Format Declaration

The **format declaration** is an optional first line in a kaiv text indicating:

- the type of kaiv text: data (`!kaiv`) or schema (`!kaivschema`) — Section V,
- the version (Section 9.2),
- whether the schema is strict (`:strict` , Section **??**) or relaxed (default, Section 21).

These features are presented progressively throughout the next few sections.

9.1 General Format Declaration Syntax

Data texts:

```
"!kaiv" WSP version
```

Schema definition:

```
"!kaivschema" [":strict"] WSP version
```

9.2 Version Declaration Syntax

If present, the version declaration MUST appear on the first line of the kaiv text. The syntax is:

```
"!kaiv" WSP version
```

where `version` is a valid kaiv version identifier (`1.0` or `1.1`), for example:

```
!kaiv 1.1
```

The version declaration is OPTIONAL. If omitted, parsers SHOULD parse the kaiv text using the latest 1.x version they support.

9.3 Version Declaration Errors

- A parser encountering an invalid version format MUST raise an `INVALID_VERSION_ERROR`. Valid version formats follow the semver structure [9].
- A parser encountering a version that is in valid format, but not implemented by the parser MUST raise an `UNSUPPORTED_VERSION_ERROR`
- A version declaration placed in a kaiv text on any line but the first MUST be parsed as a metadata line (Section 12), raising an `INVALID_DIRECTIVE_ERROR`.

10 Quoted Keys

Starting with version 1.1, kaiv supports two forms of keys:

1. **bare keys**: POSIX-style identifiers, identical to the kaiv 1.0 key definition (ASCII letters, digits, and underscores, starting with a letter or underscore),
2. **quoted keys**: keys enclosed in double quotes, allowing characters that would otherwise be invalid in an identifier.

Bare and quoted keys may be intermixed freely within the same kaiv text.

10.1 Quoted Key Syntax

A quoted key is a double-quoted string placed where a key would normally appear:

```
"Content-Type"=application/json
"x-request-id"=abc-123
```

The following rules apply to quoted keys:

- Quoted keys MUST contain at least one character (empty quoted keys are not allowed).
- Any character is permitted inside a quoted key except the double quote character (`U+0022`).
- No escape sequences are defined for quoted keys.

10.2 Examples

```
!kaiv 1.1

# Unquoted keys (POSIX identifiers, unchanged from 1.0)
HOST=localhost
PORT=5432
is_offline=true

# Quoted keys (any characters except double quote)
"Content-Type"=application/json
"x-forwarded-for"=192.0.2.1
"Accept-Language"=en-US,en;q=0.9

# Intermixed in the same file
DEBUG=false
"feature.dark-mode"=enabled
```

11 kaiv Comment Syntax

Starting with version 1.1, kaiv supports two comment styles:

- legacy (shell) style: `# comment` (inherited from .env files),
- C-style: `// comment` (familiar from many programming languages).

Parsers MUST treat the two styles as semantically identical, producing the same internal representation.

Part III
Metadata

12 Metadata Entries

Type annotations (Section 13), encoding directives (Section 14), and newline alias directives (Section 15) are collectively termed **metadata lines**, and parsers may emit corresponding **metadata entries**.

When metadata entries are produced, they MUST contain:

- the line number,
- the directive kind (`type` , `b64` , or `nl`),
- the argument:
  - for type annotations: the type keyword (e.g. `int` , `bool` , `x/customerID`),
  - for encoding directives: the encoded type string, if present (e.g. `json` , `application/octet-stream`), or empty if omitted,
  - for newline alias directives: the alias string (e.g. `/` , `|||`).

Applications consuming entries may establish relationships between metadata entries and data entries based on their line numbers, and use this metadata for processing (e.g.,

type casting, base64 decoding etc.). See Appendix C for semantic error conditions related to metadata entries.

13 Type Annotations

13.1 Syntax

A **type annotation** is a metadata line starting with `!` and followed by a recognized type keyword. Type annotations indicate the intended value type of the immediately following data line. For example:

```
!int
index=1

!bool
admin=true

!dec
pi=3.14159265
```

Valid type annotations are: `!bool`, `!int`, `!hex`, `!b`, `!float`, `!dec`, `!date`, `!datetime`, `!timestamp`, `!str`, `!json`, `!xml` and `!html`. The `!str` type annotation has no effect in data texts but may be used in defining constraints on string values in schemas (Section 25).

13.2 Semantics

Type annotations provide information that applications may use when processing the following data line. A type annotation cannot be applied to more than one line. In case of multiple lines using the same type annotation, each data line must have a type annotation immediately preceding it.

Type annotation lines are considered *content lines* in the sense of Section 2.3.

13.3 Standard Type Annotations

Type annotations support the following standard types:

- `bool`: any boolean representation,
- `int`: integer,
- `hex`: hexadecimal representation of a number,
- `b`: binary representation of a number,
- `float`: floating point number,
- `dec`: decimal number (fixed precision),
- `date`: a date in ISO format (YYYYMMDD or YYYY-MM-DD),
- `datetime`: a datetime in ISO format (e.g. YYYY-MM-DDThh:mm:ss), with optional time zone information (UTC assumed if TZ absent),
- `timestamp`: a POSIX timestamp (number of seconds since the Unix Epoch),
- `str`: a string value (used in schema constraints, no effect in data texts),
- `json`: an inline JSON object,
- `xml`: an inline XML fragment,
- `html`: an inline HTML fragment.

13.4 Custom Type Annotations

Domain-specific types can use the `!x/` prefix for **custom type annotations**. Bare custom types use the same character set as encoded type strings (ASCII letters, digits, and the characters `/`, `+`, `-`, `.`, `_`):

```
!x/customer_id
```

Quoted custom types follow the same rules as quoted keys (Section 10), allowing arbitrary Unicode characters except the double quote character (`U+0022`):

```
!хидентификатор/"клиента_"
!x□□/"ID"
```

13.5 Parsing Requirements

Parsers MAY ignore type annotation lines, or MAY produce corresponding **metadata entries**. If entries are emitted out of order, metadata entries MUST be emitted.

Type annotations have no effect on parser behavior; applications consuming entries may use them for validation, type casting, or documentation.

Parsers MUST handle custom and standard type annotations equally.

13.6 Application Behavior

Applications may choose to

- cast the value to the given type as the type annotation entry and the corresponding KV entry are consumed,
- preserve them as received, deferring casting to a subsequent stage,
- cast lazily on value access, or
- apply any other suitable type annotation handling strategy.

14 Encoding Directive

The single-line value constraint presents a limitation for embedding multi-line or binary data in kaiv texts. The `!b64` encoding directive addresses this by indicating that the following data line's value is base64url-encoded.

14.1 Syntax

```
!b64 TYPE
KEY=VALUE
```

where `TYPE` indicates the encoded content type, typically:

- a UTF-8 string that may be multi-line (omitted `TYPE` argument),
- a file extension (e.g., `json`, `toml`, `md`, `html`, `xml`, `yaml`),
- a registered MIME type (e.g., `text/calendar`, `application/atom+xml`, `application/octet-stream`),

Any custom types should use either:

- the generic string type: `!b64`;
- the generic binary type: `!b64 bin`;

— or use the `!b64` directive with private types, for example:

```
!b64 x/ProductDetails
```

14.2 Base64url Variant

The `!b64` directive specifically refers to the *base64url* variant (RFC 4648 §5), which uses the URL-safe alphabet (`A-Z a-z 0-9 -_`) and omits padding characters (=) unless required for decoder compatibility.

14.3 Examples

```
!b64
MULTI_LINE_STRING=aBcde123Pcgw

!b64 json
CONFIG=eyJzdGF0dXMiOiJPSyIsImVuYWJsZWQidHJ1ZX0K

!b64 yaml
SETTINGS=LS0tCmRlYnVnOiB0cnVlCnBvcnQ6IDgwODAK

!b64 application/octet-stream
CERTIFICATE=MIIC1jCCAX4CAQAwDQYJKoZIhvcNAQEFBQA...
```

14.4 Parsing Requirements

Parsers MUST produce a metadata entry for each `!b64` line (Section 12). The base64url encoding validation, decoding and interpretation of the encoded content is the application's responsibility.

14.5 Application Behavior

Applications may choose to

- decode the value as the `!b64` entry and the corresponding KV entry are consumed,
- preserve them as received, deferring decoding to a subsequent stage,
- decode lazily on value access, or
- apply any other suitable `!b64` entry handling strategy.

15 Newline Alias Directive

The **newline alias directive** `!nl` is a metadata directive that defines a newline alias within the value of the following data line, enabling a human-readable representation of a multi-line value within a single data line. For example:

```
!nl /
basho=old pond/a frog jumps in/sound of water
```

15.1 General Syntax

The `!nl` directive MUST include an argument. The argument defines the newline alias string.

```
nl-alias-line = *WSP "!nl" WSP separator EOL
```

The newline alias (`separator`) is a non-empty string using ASCII symbols and the space character (`SP`, `U+0020`). See the formal grammar (Appendix A.1) for details.

15.2 Multi-character newline aliases

The newline alias defined in the `!nl` directive is not restricted to a single symbol character; it can be an arbitrary string of allowed symbols. For example:

```
!nl |||
snippet=x=y||true;|||z=x||true;
```

In the above example, the text contains double pipes `||`, so triple pipes are used as a newline alias.

Overlapping alias matches. The parsing of the newline aliases (handled by applications, NOT parsers) follows a simple deterministic first-match-wins scanning logic. This is best illustrated by example:

```
!nl ;;
s=one;;;two
```

Here, the first `;;` substring, which is immediately following `one` is matched as a newline alias, leaving `;two` to the second encoded line.

15.3 Newline Alias Selection

The rejection of escape sequences is a core principle in kaiv. Therefore, the alias string should be selected carefully in order to avoid matches in values that would cause unintended newline substitutions.

For example, if a value contains `/`, use `||` or `#*#` instead:

```
!nl #*#
path=usr/local/bin#*#opt/bin
```

In the above example, the value does not contain spaces, and a SP (space) character can be used as a newline alias.

```
!nl
path=usr/local/bin opt/bin
```

Note that the `!nl` directive must be followed by two spaces: the first as a separator between the directive and the alias, and the second as the alias string.

WARNING. The use of spaces as newline aliases should be used with caution in order to avoid unintended consequences:

- Beware of visually confusing tabs (HTAB) for spaces (SP). Only spaces are an allowed characters in newline aliases.
- Using double or triple spaces as newline aliases in values that otherwise contain single spaces works, but one should be careful with counts as multiple spaces are not always easy to parse visually.

Convention. The conventional way to use a newline alias is the standard newline escape `\n`, noting that in kaiv `\n` is a sequence of two characters like any other:

```
!nl \n
basho=old pond\na frog jumps in\nsound of water
```

Although it is not the most readable variant, values can be escaped to this form automatically by standard tools.

15.4 Parsing Requirements

Parsers MUST emit a metadata entry for each newline alias directive (`!nl` line). Parsers MUST validate the newline alias according to the grammar, and raise the corresponding error if the newline alias is invalid (`INVALID_NL_ALIAS_ERROR`, see Appendix B.11). Parsers MUST preserve the value verbatim; performing newline alias substitutions on the values is the application's responsibility.

15.5 Application Behavior

Applications may choose to

- interpolate the newline aliases as the newline alias entry and the corresponding KV entry are consumed,
- preserve them as received, deferring interpolations to a subsequent stage,
- interpolate lazily on value access, or
- apply any other suitable newline alias handling strategy.

If the newline alias does not appear in the value, the newline alias directive has no effect.

16 Metadata Forward Compatibility

While kaiv 1.x parsers simply emit entries (or not) for metadata lines (type annotations and encoding directives), without validation or considering the context (whether a metadata is actually followed by a data line), this behavior may change in the future. The following general principles provide guidance to ensure forward compatibility:

- The scope of a metadata line is a single data line. In order to apply the same metadata line to several data lines, a copy of the metadata line should precede each data line.
- Type annotations must be applied to values that conform to the standard string representation of the types described in Section 13.3 (e.g., ISO standard for date and time, 1/0/true/false/yes/no for booleans etc.).

Part IV
Arrays

17 Arrays

An **array** is a type of value (in a key-value pair) that is composed of a sequence of strings. Each member string of an array is called an **element**.

As kaiv never uses any special delimiters or escape characters, in order to distinguish array values from simple string values, a different assignment operator is used (Section 17.1).

Thus, **array lines**—or **KA lines** (key-array)—represent a type of data line that is distinct from the simple key-value data line (with `=` as the assignment operator) that have been dealt with exclusively so far. In order to avoid ambiguity, the latter type of data line will be referred to as a **KV line** when necessary.

17.1 Array Syntax

The KA data line syntax requires an explicit separator declared in the assignment operator. The general form is:

```
key "*" SEP "=" element1 SEP element2 SEP element3 ...
```

For example, using a semicolon as the separator:

```
even*;=2;4;6;8
```

Note that separators are element separators, not element terminators: a line ending with a separator would have an empty string as the last element. The following is thus an array of three elements: `a`, `b`, and an empty string:

```
foo*;=a;b;
```

Consecutive separators produce empty strings within the array. Here is an array with three elements and an empty string for its second element:

```
bar*;=b;;a
```

The separator `SEP` uses the same alphabet as the newline alias presented in Section 15.1: ASCII symbols and the space character (Appendix A.1).

For example, using a multi-character separator:

```
even*|||=2|||4|||6|||8
```

Convention. When array elements are known not to contain semicolons, `*;=` is a conventional separator choice. For elements that may contain semicolons, choose a separator that does not appear in any element.

17.2 Whitespace Handling

As always, kaiv syntax tolerates whitespace at the start of the line and between the key and the assignment operator.

```
    even                *;=2;4;6;8
    even               *,=2,4,6,8
    nombres_decimaux   */=1,0/2,1/13,9
    nombres_decimaux  *||=1,0||2,1||13,9
```

Whitespace *within* the assignment operator, however, is *significant*: spaces (SP) are valid characters in the separator `SEP`. For example:

```
even             * =2 4 6 8
nombres_decimaux * =1,0 2,1 13,9
```

WARNING. The use of spaces as within separators, although visually appealing and following the shell word splitting (with the notable difference that only SP is allowed, HTAB is not), calls for a word of caution: Unlike the UNIX shell which trims trailing whitespace and collapses multiple whitespace characters in separators, kaiv is strict in preserving each character verbatim. This may result in unintended trailing empty string elements (in case of trailing spaces in a line), unintended empty elements within an array (in case of repeated SP characters), or elements unintentionally merged together or split due to mistaking a tab for a space or vice-versa.

Therefore, the following example, although perfectly valid, may be fragile if edited manually:

```
aba_words* - =aba - abacus - abaft - abandon - abase
```

Any whitespace past the `=` character of the assignment operator is, as always in kaiv, treated as literal characters, part of the value. Any whitespace around the separator is considered part of the adjacent element. In the following example, the last two elements have leading spaces:

```
abc*;=a; b; c
```

17.3 Array Entries

An **array entry** or a **KA entry** is analogous to a KV entry, except that it carries three fields: the key, the separator, and the value. Both the separator and the value MUST be preserved verbatim. Splitting the value into individual elements using the separator is the application's responsibility.

17.4 Array Parsing Requirements

- Parsers MUST emit a KA entry for each array line.
- The separator MUST be preserved verbatim as extracted from the assignment operator.
- The value MUST be preserved verbatim.
- Parsers MUST NOT split the value into elements. Splitting on the separator is the application's responsibility.

18 Arrays and Directives

Directives are distributive with respect to arrays: a directive preceding an array line is applied to each element in the array value.

For example:

```
!int
primes*;=1;2;3;5;7

!b64
encoded*;=YQo;QQo;MAo
```

Arrays are therefore *homogeneous lists*: all elements in an array share the same type annotation and encoding directive. There is no mechanism to annotate individual array elements independently.

Part V
Schemas

19 Schema Definition

A **schema** is a special text specifying the structure of data-bearing kaiv texts. When necessary to avoid ambiguity we will be referring to non-schema kaiv texts as **data texts**.

Even though schemas formally follow the key-value form, values are reduced to a boolean flag (required or not). We will refer to keys in schemas as **fields**.

Schemas files SHOULD use the `.kvs` file extension.

Structure. Although schemas mimic standard kaiv syntax to a great extent, there are several specificities that schema parsers and applications validating data texts against schemas need to be aware of:

- format declaration: different from data texts (Section 19.1),
- value restrictions: must be empty or `!` (Section 19.2),
- keys must be unique (Section 19.3),
- no `!b64` or `!nl` directives (Section 23),
- array fields use no separators (Section 24).

The formal grammar is presented in Appendix A.3.

19.1 Schema Format Declaration

Schema files use the `!kaivschema` format declaration, followed by the kaiv version, and the schema's own canonical URL. For example:

```
!kaivschema 1.1 https://example.org/schema.kvs
```

Format declarations in schemas are REQUIRED.

19.2 Schema Field Definitions

A schema text itself includes no data, only keys that define the kaiv text structure for concrete kaiv texts. This is a definition by example: the schema simply lists the keys with assignment operators and empty values. For example:

```
HOST=
PORT=
```

By default, the keys defined in the schema are not required in the data text. If the data text omits a key from the schema, applications MUST treat it as if defined with an empty value.

Required Keys. Keys can be marked as required in the schema, in which case the corresponding values in conforming data texts MUST NOT be empty.

As kaiv has no concept of null values—only empty string values—it is important to note that required fields effectively cover two meanings: *non-null* and *non-empty string* values.

Required keys are defined by setting the value in the schema to a single exclamation mark `!`:

```
!kaivschema 1.1 https://schema.example.org/server.kvs
HOST=!
PORT=
```

Here, `HOST` is required but `PORT` is not. See Appendix C.5.

Note: The `!` sentinel has special meaning only in schema definitions. In data texts, the value `!` is a literal string with no special semantics:

```
!kaiv 1.1
PRIORITY=!
```

Here, `PRIORITY` has the literal value `!`.

19.3 Schemas and Key Uniqueness

Key uniqueness in schema texts. A schema MUST NOT contain duplicate fields. A schema containing duplicate keys is invalid.

Key uniqueness in data texts implementing a schema. A data text conforming to a schema MUST NOT contain duplicate keys for any field defined in the schema. Applications validating against a schema MUST reject data texts containing duplicate occurrences of schema-defined keys.

Keys *not defined* in a relaxed schema are not subject to this constraint. See Appendix C.2 and Appendix C.4.

20 Schema Declaration in Data Texts

Data texts can reference a schema using one or more `!schema` lines placed after the format declaration:

```
!kaiv 1.1
!schema https://example.org/schema.kvs
```

A data text referencing a schema is said to **implement** the referenced schema. If the schema contract is implemented correctly, we say that the data text **conforms** to the schema.

Schema lines MUST appear after the format declaration and before any content lines. Parsers SHOULD NOT validate texts against schemas — this would be inconsistent with the lightweight design of kaiv parsers. Schema validation is the responsibility of the application consuming the entry stream.

The referenced schema SHOULD be a file resolvable at the specified URL. Applications MAY implement schema caching or an aliasing system to local schema files. If a referenced schema cannot be resolved, applications SHOULD raise a `SchemaResolutionError` (Appendix C.3).

21 Relaxed and Strict Schemas

Schemas are **relaxed** by default, i.e., data texts may contain keys not defined in the schema. For example, we may define an RPC request schema to include only a `method` key:

```
!kaivschema 1.1 https://schema.example.org/rpc-request.kvs
method=!
```

An actual data text could then simply list any arguments as additional keys:

```
!kaiv 1.1
!schema https://schema.example.org/rpc-request.kvs
method=AddNumbers
x=1
y=2
```

A schema can be defined as **strict** with the `:strict` modifier:

```
!kaivschema:strict 1.1 https://example.org/schema.kvs
```

A data text is conforming to a strict schema only if it includes no other keys than the ones defined by the schema (Appendix C.6).

22 Schema Composition and Inheritance

22.1 Schema Composition

A data text may implement multiple schemas:

```
!kaiv 1.1
!schema https://example.org/base.kvs
!schema https://example.org/extension.kvs
```

Composition only makes sense for relaxed schemas: implementing a strict schema is incompatible with any fields not defined in that schema.

22.2 Schema Inheritance

The same `!schema` syntax can be used in schema definition texts in order to inherit or extend another schema:

```
!kaivschema 1.1 https://example.com/derived-schema.kvs
!schema https://example.com/base-schema.kvs
```

Using this mechanism, complex schemas can be constructed from simpler schemas without field repetition (i.e., using a normalized data structure).

```
!kaivschema 1.1 https://example.com/complex.kvs
!schema https://example.com/simple-1.kvs
!schema https://example.com/simple-2.kvs

# extend with added fields
required_field_1=!
required_field_2=!
field_3=
field_4=
```

23 Schemas and Metadata

23.1 Type Annotations and Schemas

Type annotations in a schema define the expected type of the corresponding key in conforming data texts. For example:

```
!kaivschema 1.1
!schema https://schema.example.org/server.kvs
HOST=!
!int
PORT=
!bool
DEBUG=
```

Here, `PORT` is declared as an integer and `DEBUG` as a boolean. A conforming data text need not repeat these annotations:

```
!kaiv 1.1
!schema https://schema.example.org/server.kvs
HOST=localhost
PORT=5432
DEBUG=true
```

Applications validating against the schema MAY apply schema-defined type annotations to data text entries that do not carry their own type annotations. If a data text entry carries an explicit type annotation for a key that has a schema-defined type annotation, the explicit annotation MUST match the schema-defined annotation. A mismatch means the data text is non-conforming, regardless of whether the schema is strict or relaxed.

23.2 Encoding Directives and Schemas

Encoding directives (`!b64`) MUST NOT appear in schemas. Encoding is a property of a particular value instance, not of the key's structural definition: the same key may or may not require encoding depending on the value. Encoding directives MUST be explicit in data texts.

23.3 Newline Alias Directives and Schemas

Newline alias directives (`!nl`) MUST NOT appear in schemas. As with the `!b64` directive, newline aliasing is a property of a particular value instance, not of the key's structural definition: the same key may or may not require this kind of encoding depending on the value. Newline alias directives MUST be explicit in data texts.

24 Schemas and Arrays

All the field definitions in schemas presented so far used KV lines (simple string values). Array fields (KA lines) can be included in a schema using the array assignment operator with a blank separator `*=` , for example:

```
!kaivschema 1.1 https://example.com/post.kvs
title=!
text=!
categories*=!
tags*=
```

In the above example, the schema defines two array fields, `categories` , which is required, and `tags` , which is not.

A required array field MUST have at least one non-empty element.

25 Schema Constraints

In schemas, type annotations can contain **constraints**. Schema constraints use a micro-language that should be intuitively familiar:

- choices: `{a,b,c,...}` — an exhaustive list of valid values
- open interval: `(a,b)` — a range exclusive of bounds ($x : a < x < b$)
- closed interval: `[a,b]` — a range inclusive of bounds ($x : a \leq x \leq b$)
- mixed intervals: `(a,b]` , `[a,b)` — defined analogously ($x : a < x \leq b$ and $x : a \leq x < b$, respectively)

— unbounded intervals are represented with a missing bound, e.g., `[0,)` for all positive numbers. The missing literal must be associated with an open interval end (round parenthesis `(` or `)`).

For example:

```
!kaivschema 1.1 https://example.com/schema.kvs
# even digits only
!int {0,2,4,6,8}
i=

# 0 < x < 10
!float (0,10)
x=

# any datetime in 2026
!datetime [2026-01-01T00:00:00,2027-01-01T00:00:00)
dt=
```

Constraints are only allowed in schemas, not data texts.

25.1 Constraints Syntax

Both the enumerated constraint and the range constraints use commas as literal separators. If literals are unquoted, any whitespace *is significant* and considered part of the adjacent literal.

Literals may be quoted, which is necessary in cases where they may contain commas or parentheses, brackets or braces. If literals are quoted, then whitespace within the constraint may be liberally applied. For example, if the locale uses a comma for the decimal point, then decimal values must be quoted:

```
!float ["0,5", "1,5"]
```

Quoted constraint literals follow the same rules as quoted keys (Section 10): enclosed in double quotes (`U+0022`), containing any character except the double quote character.

Union composition. Several space-separated constraints can be combined on the same line, producing a resulting constraint set that is the union of the individual constraints. For example, the following constraint allows any even digits, or any number greater than or equal to 10:

```
!float {2,4,6,8} [10,)
```

25.2 Constraints and Arrays

As with type annotations and metadata directives, constraints are distributive with respect to arrays: each element of an array must satisfy the constraint.

25.3 Ordering and Comparison

If the constrained type defines a total order (e.g., numeric types, dates, timestamps), range boundary checks SHOULD be performed on the typed (cast) values.

If the constrained type does not define a total order (e.g., a custom type prefixed with `x/`), range boundary checks SHOULD be performed on the raw string values using lexicographic order, or an application-defined collation.

25.4 Parsing Requirements

When parsing schema texts, parsers MUST validate that the constraint syntax is conformant to the grammar, and raise an `INVALID_CONSTRAINT_ERROR` (Appendix B.12) if the validation fails. Parsers MUST NOT do any further constraint validation: e.g., whether the literals are valid representations of the type, whether the lower bound is less than the upper bound, etc. Those are all application responsibilities.

25.5 Application Behavior

Prior to validating a data text against a schema, applications MUST validate the semantic integrity of the schema. In particular, applications MUST

1. verify that all literals can be cast to the given type;
2. in case of range constraints, check if the type implements a relation of total order, and use string collation order if not;
3. in case of range constraints with two literals (not unbounded), verify that the lower bound is less than the upper bound.

26 References

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2 edition, 2006.

[2] Scott Bradner. Key words for use in RFCs to indicate requirement levels. RFC 2119, March 1997. https://www.rfc-editor.org/rfc/rfc2119.html.

[3] Dave Crocker and Paul Overell. Augmented BNF for syntax specifications: ABNF. RFC 5234, January 2008. https://www.rfc-editor.org/rfc/rfc5234.html.

[4] Ned Freed and Nathaniel S. Borenstein. Multipurpose internet mail extensions (MIME) part one: Format of internet message bodies. RFC 2045, November 1996. https://www.rfc-editor.org/rfc/rfc2045.html.

[5] Ned Freed and Nathaniel S. Borenstein. Multipurpose internet mail extensions (MIME) part two: Media types. RFC 2046, November 1996. https://www.rfc-editor.org/rfc/rfc2046.html.

[6] Ned Freed, John C. Klensin, and Tony Hansen. Media type specifications and registration procedures. RFC 6838, January 2013. https://www.rfc-editor.org/rfc/rfc6838.html.

[7] IEEE Computer Society. IEEE standard for information technology—Portable Operating System Interface (POSIX) base specifications, issue 7. IEEE Std 1003.1-2017, January 2018. Section 4.28 defines the shebang (#!) syntax for interpreter scripts. https://standards.ieee.org/ieee/1003.1/6829/.

[8] Linux/BSD man-pages project. *execve(2) - execute a file*, 2023. Documents the "shebang" convention for interpreter scripts. https://man7.org/linux/man-pages/man2/execve.2.html.

[9] Tom Preston-Werner. Semantic versioning 2.0.0. https://semver.org/, 2013.

[10] The Unicode Consortium. The Unicode standard. Technical report, 2023. https://www.unicode.org/versions/Unicode15.0.0/.

[11] Henry S. Thompson and Chris Lilley. XML media types. RFC 7303, July 2014. https://www.rfc-editor.org/rfc/rfc7303.html.

Appendices

Appendix A Formal Grammar

The following grammar represents the normative definition of kaiv 1.1 syntax. In case of any ambiguities in the main text of this specification (Sections), this grammar has precedence.

A.1 Grammar

Listing 1: kaiv 1.1 grammar

```
kaiv-text    = [format-declaration] *schema-line *(content-line / blank-line)
content-line = kv-line / ka-line / type-annotation-line / nl-alias-line
             / b64-line / comment-line
; Lines
format-declaration   = "!kaiv" WSP version EOL
schema-line          = *WSP "!schema" WSP schema-url EOL
kv-line              = *WSP key *WSP "=" value EOL
ka-line              = *WSP key *WSP "*" separator "=" value EOL
type-annotation-line = *WSP "!" type EOL
b64-line             = *WSP "!b64" [WSP encoded-type] EOL
nl-alias-line        = *WSP "!nl" WSP separator EOL
comment-line         = *WSP ("#" / "//") comment-text EOL
blank-line           = *WSP EOL
; Line components
version            = "1.0" / "1.1"
schema-url         = 1*valid-char
key                = bare-key / quoted-key
bare-key           = ( ALPHA / "_" ) *( ALPHA / DIGIT / "_" )
quoted-key         = quoted-id
separator          = 1*sep-char
type               = standard-type / custom-type
encoded-type       = standard-enc-type / custom-type
standard-type      = "bool" / "int" / "hex" / "b" / "float" / "dec"
                   / "date" / "datetime" / "timestamp" / "str"
                   / "json" / "xml" / "html"
standard-enc-type = bare-type
custom-type        = "x/" (bare-type / quoted-type)
bare-type          = 1*type-char
quoted-type        = quoted-id
quoted-id          = DQUOTE 1*quoted-char DQUOTE
value              = *valid-char
comment-text       = *valid-char
; Characters
type-char          = ALPHA / DIGIT / "/" / "+" / "-" / "." / "_"
sep-char           = SP / %x21-2F / %x3A-3C / %x3E-40 / %x5B-60 / %x7B-7E
                   ; ASCII space and symbols; excludes letters, digits, and "="
quoted-char        = (%x20-21 / %x23-7E / %x80-10FFFF)
                   ; any character except DQUOTE (%x22) and control chars
valid-char         = %x01-09 / %x0B-0C / %x0E-10FFFF
EOL                = LF / CRLF
```

The rules from RFC 5234 [3] are used:

- `ALPHA = %x41-5A/ %x61-7A` (A-Z / a-z)
- `DIGIT = %x30-39` (0-9)
- `DQUOTE = %x22` (double quote)
- `SP = %x20` (space)
- `LF = %x0A` (line feed)
- `CR = %x0D` (carriage return)

— `CRLF = CR LF`
— `WSP = SP / HTAB` i.e. `WSP = %x20/ %x09` (space or tab)

A.2 Notes

Note on `valid-char`: The `valid-char` rule explicitly excludes NUL (`U+0000`), LF (`U+000A`), and CR (`U+000D`) for parsing reasons. Implementations MUST perform UTF-8 validation separately (Section 4.7). As `valid-char` includes only valid code points, the range `U+D800` to `U+DFFF` (UTF-16 surrogates) is therefore implicitly excluded.

Note on EOF: This grammar requires every line to be terminated by EOL. Input where the final line lacks an EOL terminator does not match this grammar; parsers MUST raise a `MISSING_FINAL_EOL_ERROR` (Appendix B.13). See footnote 2 on page 10 for the rationale behind this strict requirement.

A.3 Schema Definition

A schema definition conforms to data text grammars in all respects with the following divergences:

Format declaration:

— The format declaration line is required for schemas.
— Schema definitions use the `!kaivschema` format declaration syntax instead of `!kaiv`.
— The format declaration command is optionally suffixed with `:strict`.
— The schema URL (the schema's own canonical URL) is included in the format declaration.

Line types: The encoding directive `!b64` and the newline alias directive `!nl` are not allowed in schemas.

Values: All values in a schema must be either empty (assignment operator immediately followed by EOL), or equal to `!`.

Constraints: Unlike data texts, type annotations in schemas may contain constraints.

The schema grammar is thus:

Listing 2: kaiv schema grammar — modified rules

```
kaiv-text            = format-declaration *(schema-content-line / blank-line)
format-declaration   = "!kaivschema" [":strict"] WSP version WSP canonical-url EOL
canonical-url        = 1*valid-char
schema-content-line  = kv-line / ka-line / type-annotation-line / comment-line
type-annotation-line = *WSP "!" type *(WSP constraint) EOL
value = ["!"]
; Constraints
constraint        = choice-constraint / range-constraint
choice-constraint = "{" literal *("," literal) "}"
range-constraint  = range-open literal "," literal range-close
                  / "(" "," literal range-close
                  / range-open literal "," ")"
range-open        = "(" / "["
range-close       = ")" / "]"
literal           = 1*literal-char / (*WSP quoted-id *WSP)
literal-char = %x21-27 / %x2A-2B / %x2D-5A / %x5C / %x5E-7A / %x7C / %x7E-10FFFF
; visible chars except comma (,) parens () brackets [] braces {} and space
```

Appendix B Parser Errors

> *Changes since v1.0: added `INVALID_VERSION_ERROR`, `UNSUPPORTED_VERSION_ERROR`, `INVALID_DIRECTIVE_ERROR`, `INVALID_NL_ALIAS_ERROR`, and `INVALID_CONSTRAINT_ERROR`; updated `INVALID_KEY_ERROR` to cover quoted keys; added `INVALID_SEPARATOR_ERROR`.*

This appendix classifies the cases of parse failures. If feasible in the implementation environment, in case of parse failures, parsers SHOULD return one of the errors below. If possible, error reports SHOULD include line numbers (Section 4.1).

The errors below are listed in order of priority. In case of multiple errors occurring on any line, parsers MUST report the highest priority error, and MAY report further errors from the same line.

B.1 `BOM_ERROR`

A kaiv text MUST NOT begin with a UTF-8 BOM sequence (EF BB BF) [10]. Parsers MUST detect texts starting with the BOM sequence and raise a `BOM_ERROR`.

- BOM detection must occur before any other parsing action (including UTF-8 validation).
- The BOM bytes MUST NOT be interpreted as part of the kaiv text syntax.
- BOM errors occur before line parsing begins, and SHOULD be reported without line numbers.

Whether parsing proceeds after this error detection follows the implementation's error handling strategy.

B.2 `INVALID_UTF8_ERROR`

Parsers MUST process input as UTF-8. Upon encountering invalid UTF-8 sequences, parsers MUST raise an `INVALID_UTF8_ERROR`.

B.3 `INVALID_VERSION_ERROR`

When encountering a version declaration, parsers MUST parse the version as the second token on the line (the `!kaiv` command being the first).

If the version does not match the semver format `/^[0-9]+\.[0-9]+(\.[0-9]+)?$/`, parsers MUST raise an `INVALID_VERSION_ERROR`.

B.4 `UNSUPPORTED_VERSION_ERROR`

If the version is not one of: `1.0` or `1.1`, parsers MUST raise an `UNSUPPORTED_VERSION_ERROR`.

Parsers also MUST raise `UNSUPPORTED_VERSION_ERROR` if they don't implement a particular version.

Parsers implementing `1.1` MUST NOT raise `UNSUPPORTED_VERSION_ERROR` on encountering `1.0` as `1.0` files are valid under kaiv `1.1` grammar.

B.5 `INVALID_CHARACTER_ERROR`

A kaiv text MUST NOT contain standalone CR characters (`U+000D`) or NUL characters (`U+0000`). Parsers MUST raise an `INVALID_CHARACTER_ERROR` when encountering:

- a CR character (`U+000D`) that is not part of a CRLF sequence
- a NUL character (`U+0000`)

B.6 EMPTY_KEY_ERROR

A key MUST NOT be an empty string or consist of whitespace characters. If a data line starts with the assignment operator optionally preceded by whitespace, parsers MUST raise an `EMPTY_KEY_ERROR`.

B.7 MISSING_OPERATOR_ERROR

If a data line (i.e. a line that is neither blank nor a comment line) does not contain an assignment operator, parsers MUST raise a `MISSING_OPERATOR_ERROR`.

B.8 INVALID_KEY_ERROR

All keys in a kaiv text MUST conform to the key patterns.

- Unquoted keys MUST:
  1. start with an ASCII letter or an underscore.
  2. contain only ASCII letters, numbers, and underscores.
- Quoted keys MUST:
  1. contain at least one character (empty quoted keys are not allowed).
  2. not contain double quote characters (`U+0022`).

Otherwise, parsers MUST raise an `INVALID_KEY_ERROR`.

B.9 INVALID_DIRECTIVE_ERROR

A line beginning with `!` (after stripping leading whitespace) that does not match any of the following productions — version declaration (`!kaiv`), schema declaration (`!kaivschema`), schema line (`!schema`), type annotation (`!` + recognized type keyword), encoding directive (`!b64`), newline alias directive (`!nl`), or custom annotation (`!x/`) — MUST raise an `INVALID_DIRECTIVE_ERROR`.

B.10 INVALID_SEPARATOR_ERROR

If the separator in a KA line's assignment operator (the string between `*` and `=` in the array assignment operator) does not match the grammar's `separator` rule (Appendix A.1), parsers MUST raise an `INVALID_SEPARATOR_ERROR`.

B.11 INVALID_NL_ALIAS_ERROR

If the newline alias string does not match the grammar's `separator` rule (Appendix A.1), parsers MUST raise an `INVALID_NL_ALIAS_ERROR`.

B.12 INVALID_CONSTRAINT_ERROR

If a constraint in a schema does not match the schema grammar's constraint rules (Appendix A.3), parsers MUST raise an `INVALID_CONSTRAINT_ERROR`.

B.13 `MISSING_FINAL_EOL_ERROR`

If the final line of a kaiv text is missing an EOL terminator, parsers MUST consider the line invalid, and MUST raise a `MISSING_FINAL_EOL_ERROR`. This error MAY be reported without a line number.

Appendix C Application Errors

Unlike parser errors (Appendix B), which address syntactic violations detected during parsing, application errors address semantic violations detected by applications consuming the entry stream. Applications conforming to kaiv 1.1 semantics SHOULD detect and report the following conditions.

This section uses the term **document** for the application-level structure constructed from entries corresponding to a kaiv text.

Nomenclature. Application errors with a `Schema`-prefix (`Schema*Error`) are inherent schema definition errors. Errors with a `SchemaError`-suffix (`*SchemaError`) are conformity errors resulting from a kaiv text violating schema restrictions.

C.1 `MetadataWithoutTargetError`

Applications detecting a metadata entry that is not immediately followed by a KV or KA entry in line number order—whether followed by another metadata line, a blank line, a comment line, or EOF —SHOULD raise a `MetadataWithoutTargetError`.

C.2 `SchemaDuplicateKeyError`

Applications detecting a duplicate key in a schema definition (kaiv text with `!kaivschema` format declaration) SHOULD raise a `SchemaDuplicateKeyError`.

C.3 `SchemaResolutionError`

Applications attempting to validate a document against a schema referenced in the format declaration SHOULD raise a `SchemaResolutionError` if the schema cannot be retrieved from the specified URL.

C.4 `DuplicateKeySchemaError`

Validating a document against a schema, applications SHOULD raise a `DuplicateKeySchemaError` if a document has two KV or KA entries for a field defined in the schema. Applications SHOULD NOT raise this error in case of fields not defined in a relaxed schema.

C.5 `RequiredKeySchemaError`

Validating a document against a schema, applications SHOULD raise a `RequiredKeySchemaError` if a document does not have an entry for a field that is required in the schema, or if this entry's value is empty (cf. Section 19.2).

C.6 `UndefinedFieldStrictSchemaError`

Validating a document against a strict schema, applications SHOULD raise an `UndefinedFieldStrictSchema` if a document includes an entry for a field that is not defined by the schema.

Appendix D Variable Interpolation (Non-Normative)

> *This appendix is non-normative. It provides a recommendation for applications that choose to support variable interpolation in kaiv values. Nothing in this appendix changes the behavior of kaiv parsers or imposes conformance requirements on applications.*

D.1 Parser Behavior

The `$` character (`U+0024`) has no syntactic significance in kaiv. Parsers MUST NOT treat it specially: it is an ordinary character in values under the `valid-char` rule. Parsers MUST emit values containing `$` verbatim, without modification.

Any variable expansion is performed by the application after consuming the entry stream.

D.2 Recommended Syntax

Applications that support variable interpolation SHOULD use a subset of POSIX parameter expansion syntax as defined in IEEE Std 1003.1 [7], §2.6.2 (Parameter Expansion).

The recommended subset consists of:

1. **Simple expansion:**

```
"$" IDENTIFIER
```

2. **Braced expansion:**

```
"${" IDENTIFIER "}"
```

3. **Default value:**

```
"${" IDENTIFIER ":-" default-value "}"
```

where `IDENTIFIER` conforms to the kaiv key rule:

```
IDENTIFIER = ( ALPHA / "_" ) *( ALPHA / DIGIT / "_" )
```

For example:

```
!kaiv 1.1
DATABASE_URL=postgres://$DB_USER:${DB_PASS}@${DB_HOST}:${DB_PORT:-5432}
```

Applications SHOULD NOT implement the full POSIX parameter expansion syntax. In particular, the following POSIX forms are not recommended for use in kaiv values:

- `${VAR:+alternate}` (alternate value),
- `${VAR:?message}` (error if unset),
- `${#VAR}` (string length),
- substring expansion, pattern removal, and all other POSIX parameter expansion operators.

D.3 Variable Resolution

This appendix does not prescribe a variable resolution strategy. Applications SHOULD document:

1. *the source of variable values:* environment variables, entries defined earlier in the same kaiv text, an external context object, or some combination thereof;
2. *the resolution order* when multiple sources are available (e.g., whether same-file definitions override environment variables or vice versa);
3. *the behavior for undefined variables:* expansion to an empty string, an error, or literal passthrough (i.e., the unexpanded text is preserved in the value).

D.4 Interaction with Encoding Directives

If a value is base64url-encoded (preceded by a `!b64` directive, Section 14), applications SHOULD NOT perform variable interpolation on the encoded value. Interpolation, if desired, SHOULD be performed on the decoded content after base64url decoding.

D.5 Interaction with Newline Alias Directives

If a value uses a newline alias (preceded by a `!nl` directive, Section 15), applications SHOULD perform variable interpolation after newline alias substitution, so that variable references spanning an alias boundary are not inadvertently split.

D.6 Interaction with Arrays

When a KA line (Section 17) uses variable interpolation, applications SHOULD perform variable expansion on individual elements after splitting the value on the separator, not on the unsplit value. This prevents a variable whose value contains the separator character from unintentionally producing additional array elements.

For example, given:

```
HOSTS*;=${PRIMARY};${SECONDARY}
```

if `PRIMARY` expands to `a.example.com` and `SECONDARY` expands to `b.example.com`, the result is a two-element array. If expansion were performed before splitting, a `PRIMARY` value containing `;` would silently produce extra elements.

D.7 Schemas and Variable Interpolation

Variable references MUST NOT appear in schema definitions. Schema values are structural declarations (`=` for optional keys, `=!` for required keys), not data, and variable interpolation has no meaningful semantics in that context.

License

This specification is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). A human-readable summary of the license terms is provided below, but the full legal code is the authoritative version.

1 Summary

You are free to:

- **Share** — copy and redistribute the material in any medium or format
- **Adapt** — remix, transform, and build upon the material for any purpose, even commercially

Under the following terms:

- **Attribution** — You must give appropriate credit to kaiv.io, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- **No additional restrictions** — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Notices:

- You do not have to comply with the license for elements of the material in the public domain or where your use is permitted by an applicable exception or limitation.
- No warranties are given. The license may not give you all of the permissions necessary for your intended use. For example, other rights such as publicity, privacy, or moral rights may limit how you use the material.

2 Attribution Practice

When attributing this specification, please include:

- The title: "kaiv 1.1"
- The version: "1.1 RC5" (or the specific version you are referencing)
- A link to the canonical URL: `https://kaiv.io/spec/1.1-rc5.pdf`
- A copyright notice: "© 2026 the kaiv authors"

Example attribution:

```
kaiv 1.1 RC5. © 2026 the kaiv authors.
https://kaiv.io/spec/1.1-rc5.pdf (CC BY 4.0)
```

3 Disclaimer

This specification is provided "AS IS" without warranty of any kind. The authors and copyright holders make no representations or warranties, express or implied, regarding the accuracy, completeness, or suitability of this specification for any particular purpose.

In no event shall the authors or copyright holders be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with this specification or the use or other dealings in this specification.

4 Scope

This license applies to the specification document only. Any related implementations, example code, and other associated materials are licensed separately as indicated in their respective repositories.